- Think and review like: - senior software architect - performance engineer - reliability engineer - code reviewer - product-minded technical lead - Be methodical, phased, incremental. - Prioritize correctness and maintainability over speed. - Preserve user-facing behavior unless explicitly approved. We are going to plan what to do after you examine everything. Parts 1 and 2 ARE PRIORITY. The Math Engine and Zooming/Scrolling in Desk Map and Flow Chart Mandatory instruction precedence: 1) System/developer/user chat instructions 2) AGENTS.md 3) PROJECT_CONTEXT.md Before any architectural decisions: - Read PROJECT_CONTEXT.md. Hard constraints: - Work only inside this repository. - Do not modify files yet until full analysis is complete. - No PR creation. - Tie every recommendation to specific files/lines. - Avoid vague advice. - Preserve existing workflows unless change is required and called out. Primary goals (in this order): 1. Audit/verify mineral ownership math correctness. 2. Diagnose zoom/scroll performance issues on large datasets. 3. Complete technical audit: architecture, performance, tooling, scalability, reliability, security. Known current architecture facts: - Main runtime is monolithic: src/app.jsx (~3.4k LOC). - Runtime boot uses bundled dist/app.js from index.html. - Math logic currently lives inside src/app.jsx, not a separate pure domain module. - Persistence/hydration normalization in src/workspaceDomain.js. - Validation scripts in scripts/*.js. - Service worker currently self-unregisters (sw.js), while USER_MANUAL still claims app-shell caching. Known high-priority findings to verify and refine: - Math engine is not fully guarded against invalid states: - Share can exceed parent remaining interest in convey/attach/rebalance/precede flows. - Some paths clamp parent after the fact, allowing over-allocation. - Direct edits allow overriding derived fractions. - Non-finite/negative input handling is inconsistent. - Zoom/scroll lag root causes: - Full React re-render on high-frequency pointer/wheel updates. - O(E*N) edge lookup pattern in flow rendering (`find` per edge endpoint). - No virtualization/windowing for large lists/graphs. - Flow view uses two scale layers and fit logic can compound scale. - Functional issues: - Research Hub partially disabled with placeholder-only UI. - Flow edge delete path appears to call node-delete logic. - Some UX and docs are stale/misaligned with current runtime behavior. - Tooling gaps: - No repo-level lint/typecheck/CI config visible. - Tests are script-based and mostly validate derivations, not interactive render performance. Required deep-dive scope: PRIORITY******PART 1 — Math engine audit - Locate every file/function involved in ownership math: - ownership percentage computation - fractional propagation through parent-child conveyance chains - branch correction paths (precede, attach, rebalance) - aggregate totals logic - Verify multiplication semantics: I_final = I_original * f1 * f2 * f3 * ... - Identify where totals can exceed 100%. - Check precision/rounding/non-finite handling. - Check cycle/parent integrity and normalization assumptions. - Output: 1) exact files 2) current approach 3) correctness verdict 4) failure cases 5) safest fix 6) redesign recommendation or not ********PRIORITYPART 2 — Zoom/scroll root-cause diagnosis (highest practical urgency after math correctness) - Map full interaction pipeline: - components, listeners, state transitions, rendering path - Determine smallest safe fix, moderate refactor, and architecture redesign option. PART 3 — Performance review - Identify bottlenecks: - expensive recalcs - render churn - inefficient loops/data structures - blocking main-thread work - large DOM/SVG costs - Prioritize by impact. PART — Stack review - Evaluate framework/libs appropriateness and dependency necessity. PART — Tooling review - Testing, linting, formatting, typing, profiling, build, state management, monitoring, CI/CD, logging. Validation requirements: - Run relevant checks and report exact commands/results. - At minimum: - npm run test:smoke - npm run test:storage - npm run test:math-regression - npm run test:title-scenarios - npm run test:cross-surface - npm run test:perf Output format (before any edits): A. Executive Summary B. Math Engine Assessment C. Functional Issues D. Architecture Assessment E. Performance Assessment F. Zoom and Scroll Root Cause Analysis G. Stack / Library / Tooling Assessment H. Code Quality Assessment I. Testing / Reliability Assessment J. Security Assessment K. Prioritized Action Plan Action plan must include: - immediate fixes - high-value refactors - optional improvements - long-term redesign ideas We are

going to plan what to do after you examine everything. Parts 1 and 2 ARE PRIORITY. The Math Engine and Zooming/Scrolling in Desk Map and Flow Chart

Worked for 9m 46s

A. Executive Summary

- Read-only audit only; no files were modified. The current branch's math model is directionally correct but not safety-complete: live commit handlers can create over-allocation even though the regression scripts pass, because the scripts duplicate/cap logic instead of exercising the production handlers (src/app.jsx (line 862), src/app.jsx (line 1094), scripts/math-regression-checks.js (line 152), scripts/title-scenario-checks.js (line 41)).
- Zoom/scroll lag is real and explained by full React rerenders on high-frequency camera updates, whole-scene DOM rendering, O(E*N) edge endpoint lookup, and a compounded flow-fit scale bug (src/app.jsx (line 1978), src/app.jsx (line 2145), src/app.jsx (line 1834), src/app.jsx (line 2807), src/app.jsx (line 2839)).

B. Math Engine Assessment

- Exact files: production math lives in src/app.jsx (line 774), src/app.jsx (line 813), src/app.jsx (line 849), src/app.jsx (line 862), src/app.jsx (line 1028), src/app.jsx (line 1062); hydration/normalization is in src/workspaceDomain.js (line 63); aggregate ownership health is in src/app.jsx (line 342).
- Current approach: conveyance share is `basis * numerator/denominator` or fixed/all, and correction flows rescale the affected branch multiplicatively with `new = old * scaleFactor`; for valid inputs, that matches `I_final = I_original * f1 * f2 * ...` (src/app.jsx (line 862), src/app.jsx (line 849), src/app.jsx (line 1028)).
- Correctness verdict: the math semantics are acceptable; the invariant enforcement is not. The engine is unsafe because it clamps the parent after mutation instead of bounding the requested share before mutation (src/app.jsx (line 826), src/app.jsx (line 1048), src/app.jsx (line 1096), src/app.jsx (line 1112)).
- Failure cases: over-allocation on convey/attach, predecessor/rebalance exceeding available parent interest, direct override of derived fractions, zero/invalid denominator and non-finite inputs, and no cycle quarantine during hydration/import (src/app.jsx (line 867), src/app.jsx (line 3206), src/app.jsx (line 3363), src/workspaceDomain.js (line 110), src/app.jsx (line 1508)).
- Safest fix: extract the math into a pure domain module, validate every mutation before commit, cap requested interest against available ownership, make retained share read-only for derived nodes, and quarantine cyclic/invalid parent links instead of letting them reach runtime state.
- Redesign recommendation: no full mathematical redesign. Do redesign ownership math as a pure module boundary so the UI becomes a consumer rather than the source of truth.

C. Functional Issues

- Confirmed: Flow edge delete is wired to node deletion semantics. Edge click calls `deleteSelectedFlowElement(edge.id)`, but the function ignores the argument and deletes the currently selected node instead (src/app.jsx (line 2154), src/app.jsx (line 2120)).
- Confirmed: Research Hub is intentionally placeholder-only right now, with most of the implementation gated behind `false &&` blocks (src/app.jsx (line 2528), src/app.jsx (line 2553), src/app.jsx (line 2586)).
- Confirmed: runtime offline behavior and docs are misaligned. The manual still promises service-worker app-shell caching, while `index.html` unregisters service workers/caches on load and `sw.js` unregisters itself on activate (USER_MANUAL.md (line 174), index.html (line 140), sw.js (line 5)).

D. Architecture Assessment

- The main runtime is still a monolith: `App` owns UI, math, persistence orchestration, flow rendering, and desk-map rendering in one file (src/app.jsx (line 182)). The worst coupling is duplicated state between `nodes` and `deskMaps`, synchronized via effects on every active-map change and viewport update (src/app.jsx (line 421), src/app.jsx (line 455)).
- The repo does have the right seam for persistence normalization in src/workspaceDomain.js (line 226) and a bundle entrypoint in src/browser-entry.js (line 1), so this is a modularization job, not a framework rewrite. One prior audit item is stale: bootstrap now does have `try/catch` recovery (src/app.jsx (line 567)).

E. Performance Assessment

- Desk Map bottlenecks: `buildTree()` is O(n^2) because it filters `nodes` at every recursion step, the whole tree is rendered every frame, and camera changes bounce through `deskMaps` synchronization (src/app.jsx (line 1508), src/app.jsx (line 1536), src/app.jsx (line 2658), src/app.jsx (line 461)).
- Flow Chart bottlenecks: every camera/node move updates top-level state, rerenders the whole flow scene, and recomputes source/target lookups with `find` per edge; there is no viewport culling/windowing (src/app.jsx (line 1978), src/app.jsx (line 2024), src/app.jsx (line 2064), src/app.jsx (line 2145), src/app.jsx (line 2174)).
- The existing perf script is useful but limited: it benchmarks derived fixture transforms, not the interactive render path, so it will not catch the wheel/pointer lag the user is seeing (scripts/perf-benchmark-5x200.js (line 27), scripts/perf-benchmark-5x200.js (line 109)).

F. Zoom and Scroll Root Cause Analysis

- Desk Map pipeline: pointer/wheel handlers mutate `pz`, then an effect writes `pz` back into `deskMaps`, then another effect rehydrates `nodes/pz` from `deskMaps`, and the full recursive tree rerenders (src/app.jsx (line 1530), src/app.jsx (line 1550), src/app.jsx (line 421), src/app.jsx (line 455), src/app.jsx (line 2658)).
- Flow pipeline: pointer/wheel handlers mutate `flowPz`, `flowNodes`, or `mousePos`, which rerenders `renderTree(true)`; that render then performs edge endpoint scans and paints all nodes/edges every frame (src/app.jsx (line 1978), src/app.jsx (line 2016), src/app.jsx (line 2064), src/app.jsx (line 2138)).
- Confirmed compounded-scale bug: `safeFlowCanvasScale` multiplies `flowPz.scale * treeScale`, `fitFlowToView()` writes the same fit factor into both values, and render applies both transforms, so fit zoom gets squared on screen (src/app.jsx (line 306), src/app.jsx (line 1847), src/app.jsx (line 1853), src/app.jsx (line 2807), src/app.jsx (line 2839)).
- Smallest safe fix: fix edge deletion, add `flowNodeById`, stop double-scaling `fitFlowToView`, and only persist viewports on interaction end. Moderate refactor: split canvases into dedicated components with local transient camera refs, adjacency maps, and visible-node/edge culling. Architecture redesign: only if the target is materially beyond the current 5x200 scale.

G. Stack / Library / Tooling Assessment

- The stack is small and appropriate: React, ReactDOM, PapaParse, esbuild, no obvious dependency bloat (package.json (line 6)). I would not add a graph library before fixing the current hot paths.
- The offline story is weak even before the service-worker unregister logic because `index.html` still depends on remote Tailwind and Google Fonts (index.html (line 10), index.html (line 15)).
- Tooling is thin: `package.json` only exposes build/test scripts, and a repo scan found no repo-level ESLint, Prettier, TS, or CI config.

H. Code Quality Assessment

- `src/app.jsx` is too large to reason about safely, and the current state-sync pattern hides behavior changes behind effects instead of explicit domain calls (src/app.jsx (line 182), src/app.jsx (line 421), src/app.jsx (line 455)).
- Test drift risk is high because the math scripts reimplement production helpers instead of importing them (scripts/math-regression-checks.js (line 7), scripts/title-scenario-checks.js (line 41)). Also, the repo README is effectively empty (README.md (line 1)).

I. Testing / Reliability Assessment

- Validation run and result: `npm run test:smoke` PASS, `npm run test:storage` PASS, `npm run test:math-regression` PASS, `npm run test:title-scenarios` PASS, `npm run test:cross-surface` PASS, `npm run test:perf` PASS, and `npm test` PASS.
- Latest perf output: `load_fixture 1.47ms`, `derive_runsheet_all 0.20ms`, `derive_ledger_views_all 10.63ms`, `derive_flow_all 0.29ms`, warnings `0`. Reliability gap: those checks do not hit the live handlers in src/app.jsx (line 1062) or browser frame-time behavior.

J. Security Assessment

- Highest concrete issue: imported CSV `INTERNAL_DOC` is trusted into `docData`, then rendered in `<object>` and opened in a new tab without validation. That allows arbitrary URL targets from imported workspaces (src/app.jsx (line 1317), src/app.jsx (line 3448), src/app.jsx (line 3452)).
- Secondary issues: `target="_blank"` lacks `rel`, startup failure UIs use raw `innerHTML`, and IDs fall back to `Math.random()` when `crypto.randomUUID` is unavailable (src/app.jsx (line 7), index.html (line 115), src/workspaceStorage.js (line 27), src/auditLog.js (line 28), src/syncEngine.js (line 28)).

K. Prioritized Action Plan

Plan

LANDroid Phase Plan: Math Hardening First, Camera/Canvas Second

Summary

- Keep the current React/esbuild stack and current workflows, but move ownership math out of UI handlers into a pure domain module.
- Fix Desk Map and Flow Chart camera/render hot paths without expanding feature scope.
- Defer any library adoption or renderer rewrite unless the moderate refactor fails on the current large fixture.

Immediate Fixes

- Add a pure ownership module, recommended at `src/domain/ownershipMath.js`, exporting `validateOwnershipGraph`, `calculateConveyedShare`, `applyConveyance`, `applyAttachConveyance`, `applyPredecessor`, `applyBranchRebalance`, and `summarizeOwnershipByRoot`; replace the math in src/app.jsx (line 774), src/app.jsx (line 813), src/app.jsx (line 849), src/app.jsx (line 862), src/app.jsx (line 1028), and src/app.jsx (line 1062).
- Enforce invariants before mutation: finite numbers only, `share <= parent.fraction` for convey/attach, `newInitialFraction <= oldInitialFraction + parent.fraction` for precede/rebalance/edit, and no cycles/missing parents. On hydration/import, quarantine

structural failures to `parentId: 'unlinked'` and emit an audit warning from src/workspaceDomain.js (line 63) instead of letting invalid graphs reach runtime state.

- Remove freeform retained-share overrides for derived conveyance nodes by replacing the editable field at src/app.jsx (line 3363) with read-only preview text; keep edits to `initialFraction` routed through the rebalance/predecessor engine.
- Fix the Flow edge delete path by separating `deleteFlowEdge(edgeId)` from node deletion and replacing the incorrect call site at src/app.jsx (line 2154) / src/app.jsx (line 2120).
- Restrict imported/viewed document URLs to safe PDF `data:` or `blob:` URLs before assigning `viewerData` from src/app.jsx (line 1317) and before rendering src/app.jsx (line 3448).

High-Value Refactors

- Split `src/app.jsx` into at least three internal boundaries aligned with the ADR: `DeskMapCanvas`, `FlowChartCanvas`, and the new ownership domain module; keep persistence in `src/workspaceDomain.js` and `src/workspaceStorage.js`.
- Replace per-render linear scans with memoized lookup maps (`flowNodeById`, `childrenByParent`) for src/app.jsx (line 1508), src/app.jsx (line 1783), and src/app.jsx (line 2145).
- Treat `flowPz.scale` as the interactive camera zoom and `treeScale` as print/layout scale only; `fitFlowToView` at src/app.jsx (line 1834) must stop writing the same fit factor into both layers used at src/app.jsx (line 2807) and src/app.jsx (line 2839).
- Stop syncing Desk Map viewport into `deskMaps` on every pointer frame; keep transient pan/zoom in refs/state during interaction and commit once on pointer-up/wheel-settle, removing the extra sync loop at src/app.jsx (line 421) and src/app.jsx (line 455).
- Add simple viewport culling for visible Flow nodes/edges and Desk Map branches before rendering src/app.jsx (line 2145) and src/app.jsx (line 2658); do not add a virtualization dependency unless profiling still shows frame drops after lookup/camera fixes.

Tests And Acceptance

- Replace duplicated math helpers in scripts/math-regression-checks.js (line 1) and scripts/title-scenario-checks.js (line 41) with imports from the new ownership module so the scripts exercise production logic.
- Add deterministic cases for over-allocation rejection, zero/negative/NaN/Infinity handling, cycle quarantine, multi-root aggregate reporting, and invalid imported `docData`.
- Add a perf script for interactive helpers instead of fixture derivations only: `fitFlowToView`, flow edge projection with lookup maps, Desk Map tree build, and visible-node culling on the 5x200 fixture.
- Preserve the existing validation chain: `npm run test:smoke`, `npm run test:storage`, `npm run test:math-regression`, `npm run test:title-scenarios`, `npm run test:cross-surface`, `npm run test:perf`, and `npm test`.

Optional Improvements And Long-Term Direction

- Reconcile docs/runtime by either removing offline-cache language from USER_MANUAL.md (line 172) or restoring a real service-worker strategy that matches sw.js (line 5) and index.html (line 140).

- Add repo-level lint, formatter, and CI entrypoints by extending package.json (line 6); keep TypeScript optional, but add JSDoc or `@ts-check` to the new domain module if full TS is still out of scope.
- Only consider a dedicated graph/canvas library or custom canvas/WebGL renderer after the lookup-map, camera, and culling refactor is measured on the current large fixture and still misses the target interaction quality.

Assumptions

- No new dependencies in the immediate phases.
- Existing persisted workspaces remain loadable; new guards prevent additional invalid states rather than silently rewriting user data beyond quarantining structurally unsafe links.
- Research Hub remains out of scope until math and canvas hardening are complete.

Implement plan

Worked for 35m 2s

Summary

I implemented the hardening/refactor phase without touching unrelated local changes. The main changes were: moving ownership math into a pure module, routing runtime mutations through that module, quarantining invalid parent/cycle states during hydration/import, sanitizing document URLs, fixing Flow edge deletion, and landing the canvas performance refactor with lookup maps, culling helpers, component boundaries, and the flow fit-scale correction.

The biggest behavioral correction is attach-conveyance math: the requested attach share now means total attached branch interest, not just the root node's retained fraction. That fixes the prior over-allocation bug when attaching a branch with descendants.

Why

The old runtime still allowed invalid math states from UI handlers, and the flow/desk canvases were paying unnecessary work on every interaction frame. This change set moves correctness rules into one source of truth and removes the most direct rendering bottlenecks without changing the stack or adding dependencies.

Files Changed

- src/app.jsx
- src/workspaceDomain.js
- src/browser-entry.js
- src/domain/ownershipMath.js
- src/components/DeskMapCanvas.jsx
- src/components/FlowChartCanvas.jsx
- src/canvas/sceneVisibility.js
- scripts/math-regression-checks.js
- scripts/title-scenario-checks.js
- scripts/perf-benchmark-5x200.js
- dist/app.js

Validation

- `npm run build` — PASS
- `npm run test:smoke` — PASS
- `npm run test:storage` — PASS
- `npm run test:math-regression` — PASS

- `npm run test:title-scenarios` — PASS
- `npm run test:cross-surface` — PASS
- `npm run test:perf` — PASS
- `npm test` — PASS

`npm run test:perf` final timings:

- `load_fixture` 1.35ms
- `hydrate_fixture` 3.01ms
- `desk_tree_build_active` 0.10ms
- `visible_desk_roots_active` 0.04ms
- `flow_layout_all` 1.11ms
- `compute_visible_flow_scene_all` 0.30ms
- `project_visible_edges_all` 1.57ms
- `fit_flow_to_view_all` 0.15ms
- warnings: `0`

Risks / Assumptions / Follow-up

- Attach-conveyance semantics are intentionally stricter now; if users had been relying on the old root-only attach behavior, they will see corrected numbers.
- Desk Map now persists viewport on interaction end / wheel settle instead of every frame. That is intentional to reduce churn.
- I did not touch the optional docs/runtime mismatch work, lint/CI setup, or Research Hub scope.
- The repo already had unrelated local changes in files like index.html, sw.js, and others; I left those alone.

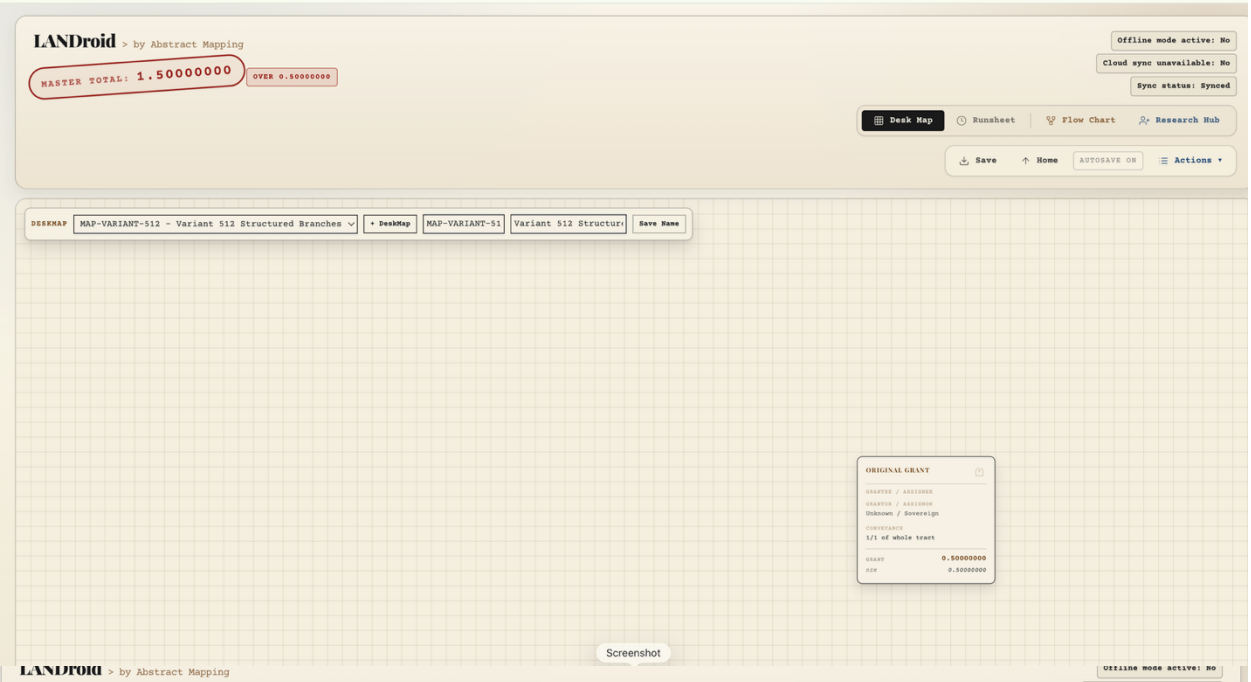
What's Next

- Manually exercise large Desk Map and Flow Chart interactions in the browser to confirm frame-time improvement on the real UI.
- Decide whether to expose attach preview text as "Attached Branch" everywhere audit/history is shown.
- Reconcile offline/service-worker docs with runtime behavior.
- Add repo-level lint/CI once this phase is accepted.

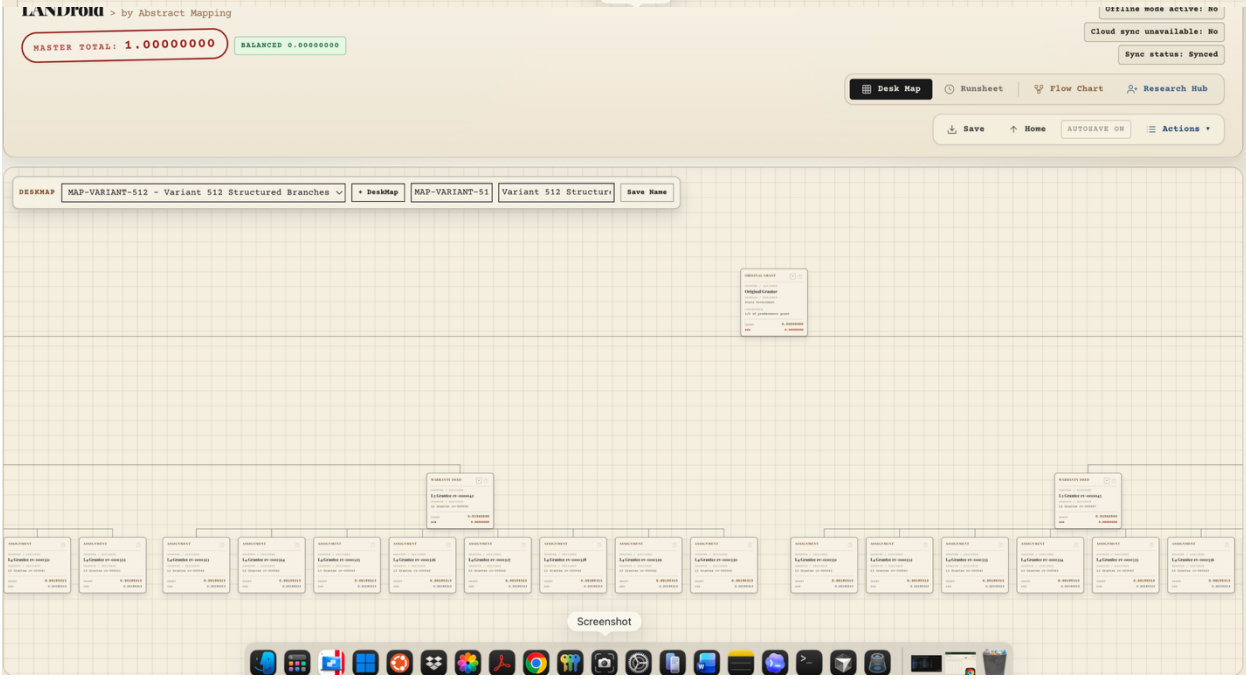
10 files changed+2140-832

Undo

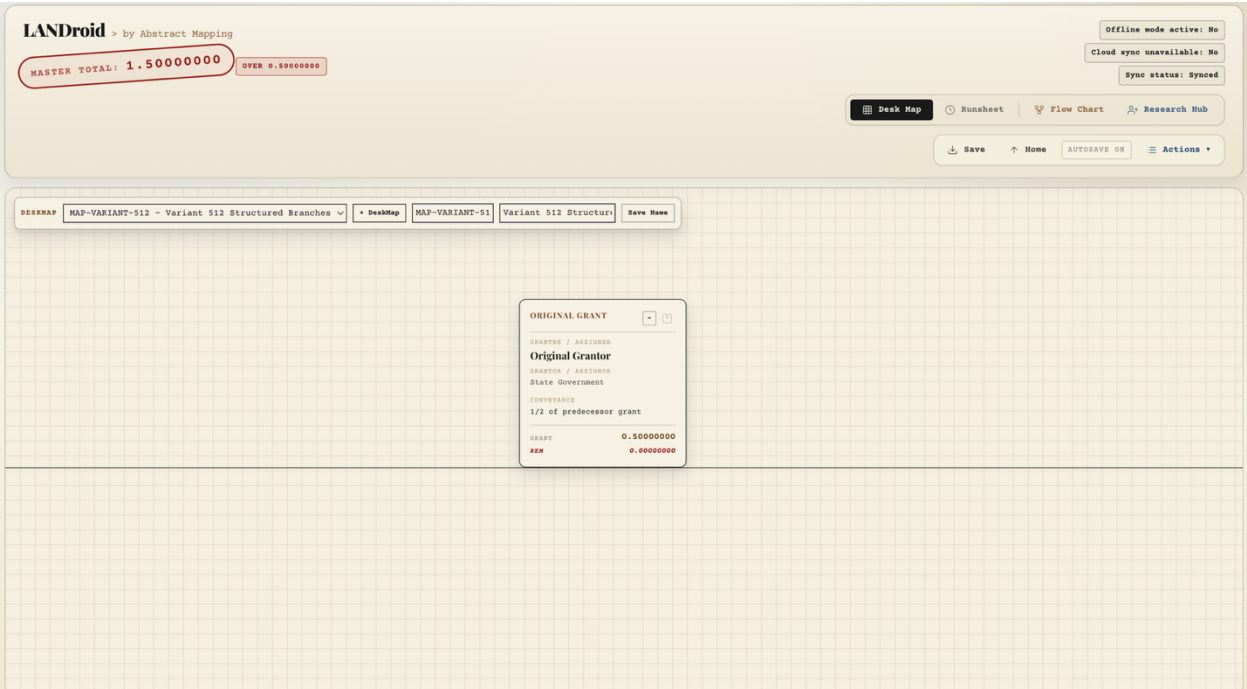
scripts/math-regression-checks.js
scripts/perf-benchmark-5x200.js
scripts/title-scenario-checks.js
src/app.jsx
src/browser-entry.js
src/canvas/sceneVisibility.js
src/components/DeskMapCanvas.jsx
src/components/FlowChartCanvas.jsx
src/domain/ownershipMath.js
src/workspaceDomain.js

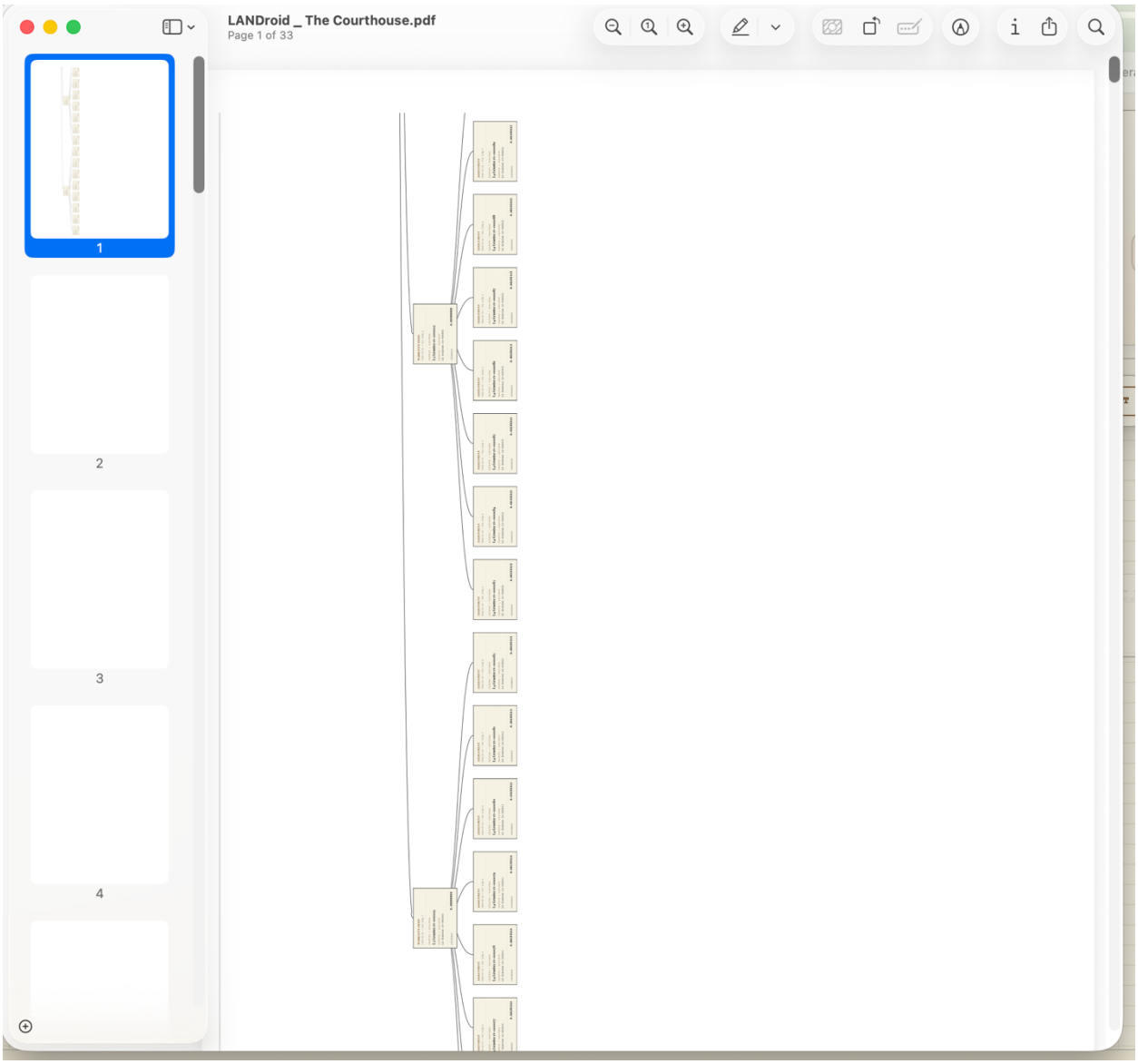


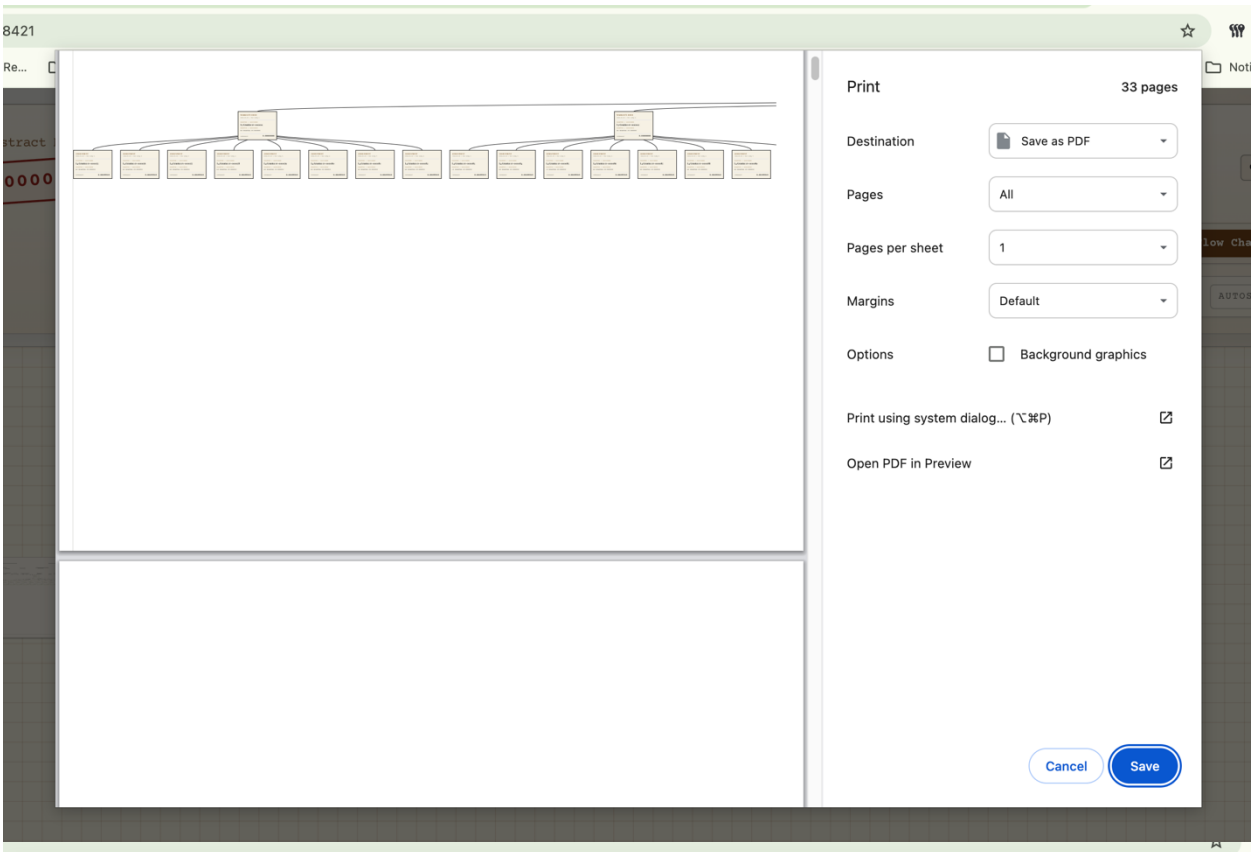
Screenshot

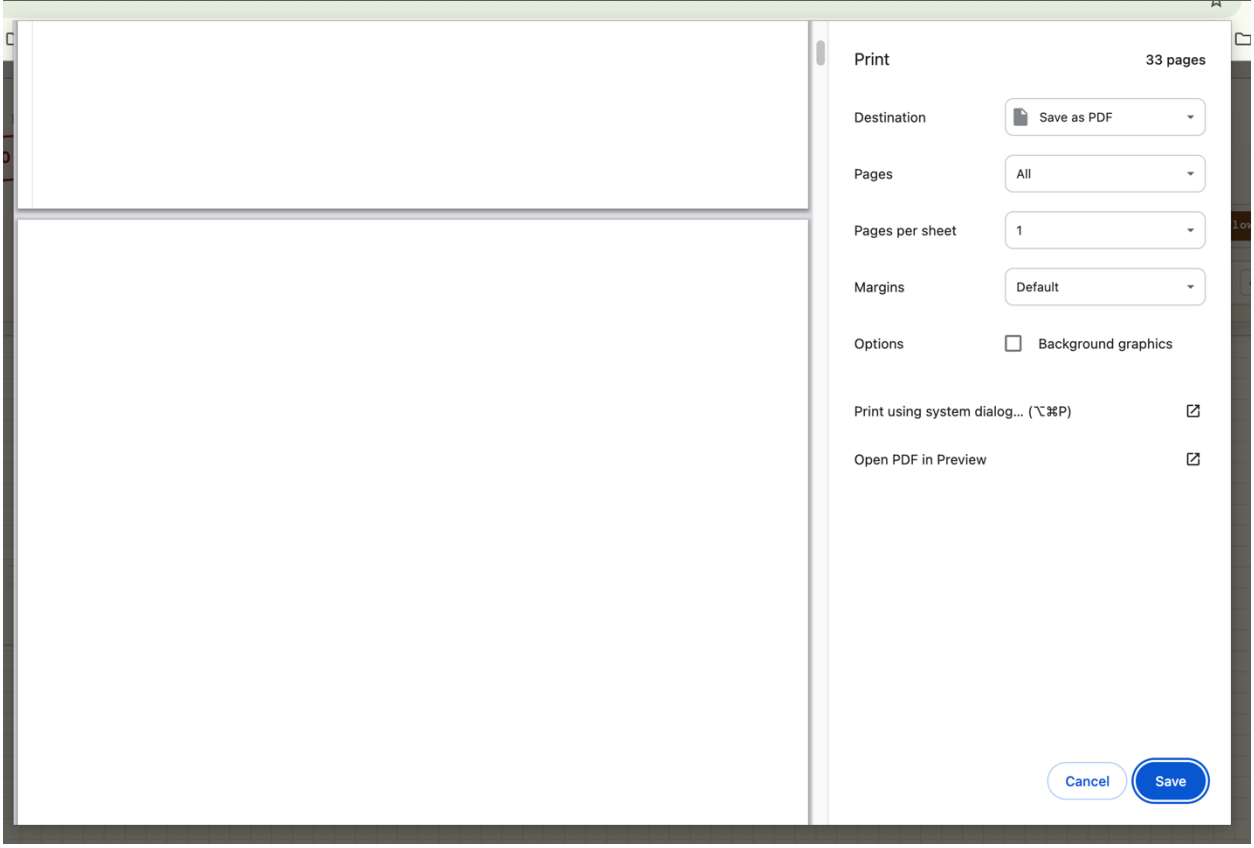


Screenshot


LANDroid > by Abstract Mapping
MASTER TOTAL: 1.50000000
OVER 0.50000000
Offline mode active: No
Cloud sync unavailable: No
Sync status: Synced
Desk Map
Runsheet
Flow Chart
Research Hub
Save
Home
AUTOSAVE ON
Actions
DESKMAP
MAP-VARIANT-512 - Variant 512 Structured Branches
+ DeskMap
MAP-VARIANT-51
Variant 512 Structur
Save Name
ORIGINAL GRANT
GRANTEE / ASSIGNEE
Original Grantor
GRANTOR / ASSIGNOR
State Government
CONVEYANCE
1/2 of predecessor grant
GRANT
0.50000000
REM
0.00000000


LANDroid _ The Courthouse.pdf
Page 1 of 33
1
2
3
4


8421
Re...
Noti
Print
33 pages
Destination
Save as PDF
Pages
All
Pages per sheet
1
Margins
Default
Options
Background graphics
Print using system dialog... (⌥⌘P)
Open PDF in Preview
Cancel
Save


Print
33 pages
Destination
Save as PDF
Pages
All
Pages per sheet
1
Margins
Default
Options
Background graphics
Print using system dialog... (⌥⌘P)
Open PDF in Preview
Cancel
Save

Well. Zooming is sooooo much better. There are some issues though. ***I have added pictures showing what I am going to describe below. 1. The math engine is still wrong. The master total doesnt update when I change values. The totals dont update either. So if i switch the original grantor who has 100% and i take him down to 50%, nothing recalculates. I also tried to add another tree and give that one the remaining 50% as a test, and it just messed everything up. THIS IS AN EXTREMELY IMPORTANT PART. I am eventually going to want to use this to calculate NPRI's And ORRI's and Working Interest for leasehold title so it has to be perfect to start. This also needs to go to AT LEAST 9 digits. 2. When i got to the flow chart, i can get it on the necessary amount of pages, but when i go to print, it only shows me the first page. The flow chart is extremely important because in the future I am going to build that out to also handle other projects. It is going to be as capable as anything on the market today. I already like the way it is set up. 3. I want the cards to also show the fractional interest as fraction too. Everything it shows now and also the fraction of what they have. Every card doesnt need to show it, but anyone who has an interest needs to have a fraction shown on top of everything else.

Worked for 20m 30s

Plan

LANDroid Phase: Exact Math, Multi-Page Flow Print, Fraction Display

Summary

- Replace the current float-based ownership engine with an exact rational core and make that core authoritative for all title math in a Desk Map.
- Treat every root chain in a Desk Map as part of one shared 100% pool; commits that would push the master total above `1.000000000` are blocked, not merely warned.
- Fix flow-chart printing by rendering each print page as its own clipped viewport over the full scene so every page paints in browser print preview and saved PDFs.
- Show current-interest fractions on interest-bearing cards in both Desk Map and Flow Chart, alongside the decimal display.
- Standardize displayed decimals to 9 places minimum; internal math stays exact and is not rounded between operations.

Key Changes

- Add a pure exact-fraction helper module using `BigInt` with normalized `{ numerator, denominator }` arithmetic, decimal parsing, reduction by GCD, comparison, scaling, add/subtract, and display formatting.
- Extend persisted node shape with string-backed rational metadata so JSON/IndexedDB/CSV can round-trip exact values without precision loss:
  - `fractionNumerator`
  - `fractionDenominator`
  - `initialFractionNumerator`
  - `initialFractionDenominator`
- Keep legacy decimal fields for compatibility, but treat rational metadata as the source of truth when present. On load, backfill missing rational fields from stored decimals exactly as entered.

- Bump workspace schema version and make hydration migrate v1 records to v2 in memory without breaking existing saves.
- In the ownership domain:
  - Recompute master total and per-root totals from exact rationals on every mutation/hydration.
  - Route all conveyance-node `initialFraction` edits, including root nodes, through rebalance logic. Root edits must no longer directly overwrite `fraction`.
  - Make conveyance `fraction` read-only in edit mode for every conveyance node, not only derived children.
  - Block `add_chain`, root rebalance, predecessor insertion, and attach operations when the resulting overall Desk Map total would exceed `1.000000000`.
  - Preserve current attach semantics fix: requested attach share means total attached branch interest.
- In the Desk Map UI:
  - Show current interest fraction text on any conveyance card with positive interest.
  - Keep the existing decimal `Grant` and `Rem` lines, but add a fraction line derived from the current retained interest rational.
  - Update all math inputs and displays from 8-digit to 9-digit precision constants in one place.
- In the Flow Chart UI:
  - Enrich flow node data with both `interestDecimal` and `interestFraction`.
  - Render the fraction on template cards only for interest-bearing conveyance nodes; do not add it to blank notes or zero-interest related docs.
  - Leave the overall visual layout intact.
- In flow printing:
  - Replace the current repeated-page negative-offset print DOM with a page-local print renderer.
  - Each page must be a `position: relative` clipped viewport with its own full-scene child translated by `(-column * pageWidth, -row * pageHeight)` and scaled once.
  - Keep interactive pan/zoom out of print rendering; print uses only layout scale and page slicing.
  - Tighten print CSS so `.print-page-break` remains the containing block and overflow clip for each page in Chromium preview and system PDF output.
- In CSV import/export:
  - Preserve existing decimal columns.
  - Add exact-rational internal columns:
    - `INTERNAL_REMAINING_NUMERATOR`
    - `INTERNAL_REMAINING_DENOMINATOR`
    - `INTERNAL_INITIAL_NUMERATOR`
    - `INTERNAL_INITIAL_DENOMINATOR`
  - Import prefers rational columns when present and falls back to legacy decimals otherwise.

Test Plan

- Add deterministic math scenarios for:
  - root edit `1.000000000 -> 0.500000000` scales descendants and leaves master total at `0.500000000`
  - adding a second root at `0.500000000` yields `1.000000000` balanced
  - adding a root that would push total above `1.000000000` is rejected
  - predecessor, attach, convey, and rebalance all preserve exact totals with 9-decimal display output
  - exact fraction display on current interest matches the rational source of truth
  - legacy workspace load without rational metadata backfills correctly
  - CSV export/import round-trips rational metadata losslessly
- Extract a pure print-slicing helper and test page offsets/slice dimensions directly so print behavior is covered without browser automation.
- Keep the existing validation chain and update it to assert 9-decimal formatting where relevant:
  - `npm run test:smoke`
  - `npm run test:storage`
  - `npm run test:math-regression`
  - `npm run test:title-scenarios`
  - `npm run test:cross-surface`
  - `npm run test:perf`
  - `npm test`
  - `npm run build`

Assumptions

- One Desk Map represents one ownership pool, so its combined root-chain total must not exceed `1.000000000`.
- Exact-rational math is implemented in-house with no new dependency.
- Existing saved workspaces and CSVs remain loadable; v2 rational fields are additive and legacy decimals remain for compatibility.
- Fraction text on cards represents current retained interest, not just the original conveyance ratio.

Expand plan

Implement plan

- Phase goal: implement exact ownership math, correct the master-total recalculation path, fix multi-page Flow Chart print output, and add fraction display on interest-bearing cards.
- In scope: domain math/persistence/UI/test updates required by that plan, plus rebuilding the browser bundle.
- Out of scope: unrelated docs/tooling cleanup, Research Hub work, or reverting any existing user changes outside this phase.

Explored 1 file

Background terminal finished with git status --short

Explored 7 files

Automatically compacting context

Error running remote compact task: unexpected status 401 Unauthorized: Your OpenAI account has been deactivated, please check your email for more information. If you feel this is an error, contact us through our help center at help.openai.com., url: https://chatgpt.com/backend-api/codex/responses/compact, cf-ray: 9db4c9f40b109381-IAH, request id: bd006bef-7bc7-441d-906b-16cb3c6384d6

Update

Fix blank screen issue

landroid

Open

Hand off

Commit

Take whatever steps are necessary to fix this.

Worked for 5m 2s

Summary

I removed the biggest remaining startup failure: LANDroid no longer depends on CDN-delivered `React`, `ReactDOM`, `Babel`, or `PapaParse` to boot. The app now builds into a local browser bundle at dist/app.js, loaded by index.html (line 181), with startup/runtime failure screens still in place in src/app.jsx (line 2).

I also added a local bundle entry at src/browser-entry.js (line 1), a repo-owned build script and runtime dependencies in package.json (line 7), kept the stale-service-worker cleanup path in index.html (line 141), and made sw.js (line 1) self-unregister instead of caching app assets.

Why It Changed

The app was still brittle because core execution depended on remote scripts. Even with viewport fixes, a failed CDN fetch or stale service worker could still produce a dead page before the app mounted. The new boot path uses local code for core runtime execution.

Files Changed

- src/browser-entry.js
- package.json
- package-lock.json
- node_modules
- index.html
- src/app.jsx
- dist/app.js
- sw.js

Validation Performed

- `npm run build` — passed
- `npm run test:smoke` — passed
- `npm run test:storage` — passed
- `npm run test:cross-surface` — passed

- `npm test` — passed
- Verified index.html no longer references `text/babel`, `unpkg.com/react`, `react-dom`, or remote PapaParse
- Verified bundle presence: dist/app.js

Risks, Assumptions, or Follow-Up Items

- `tailwindcss.com` is still remote, but it is now non-critical for boot. If it fails, styling may degrade; the app should still mount.
- An already-open tab controlled by the old service worker may need one reload to let the cleanup path run.
- dist/app.jsx is no longer on the boot path; index.html (line 181) now loads dist/app.js.

What's Next

- Localize core browser runtime so boot no longer depends on React/Babel/Papa CDNs
- Keep visible startup/runtime failure handling
- Keep stale service-worker cleanup in place
- Reload the app once in the browser so any previously active service worker can unregister
- If you still get a dead page after that reload, I want the exact browser console error next, because at that point it is no longer the old silent-blank-screen class of failure

3 files changed+23-13
Undo
index.html
package.json
src/browser-entry.js
why arent my other test files synced with what is on my computer
Worked for 32s

Because they already are synced.
Right now the only files different from `HEAD` in your working tree are:

- src/app.jsx
- dist/app.jsx
- index.html
- package.json
- package-lock.json
- src/browser-entry.js
- dist/app.js
- sw.js

Your test files under scripts are not showing as changed because they currently match what is already committed in the repo. That is why Finder/git/source control is not flagging them.
The test files that were changed earlier were already part of the prior committed fix, which is why they do not appear as modified now:

- scripts/smoke-checks.js
- scripts/cross-surface-parity-checks.js
- scripts/large-deskmap-stress.js

So the short version is:

- `scripts/*` test files are already on disk and already match the committed repo state.
- The current unsaved/uncommitted work is startup/bundling/service-worker work, not test-script work.

If you want, I can show you the exact diff between the current uncommitted state and `HEAD`, or I can commit this current change set so everything is in one coherent repo state.

README.md	Today at 9:55 AM	28 bytes	Markdov
scripts	Today at 9:55 AM	--	Folder
src	Today at 9:55 AM	--	Folder
sw.js	Today at 9:55 AM	2 KB	JavaScri
testdata	Today at 9:55 AM	--	Folder
deskmap-stress-5x200.import.csv	Today at 9:55 AM	829 KB	Comma
deskmap-stress-5x200.README.md	Today at 9:55 AM	1 KB	Markdov
deskmap-stress-5x200.summary.json	Today at 9:55 AM	1 KB	JSON
deskmap-stress-5x200.workspace.json	Today at 9:55 AM	683 KB	JSON
split-tract-imports-5x200	Today at 9:55 AM	--	Folder
upload-assorted-200.import.csv	Today at 9:55 AM	242 KB	Comma
upload-variant-512.import.csv	Today at 9:55 AM	450 KB	Comma
upload-variant-1024.import.csv	Today at 9:55 AM	850 KB	Comma
USER_MANUAL.md	Today at 9:55 AM	7 KB	Markdov

No. These

Worked for 2m 9s

That screenshot is not the same repo state I'm reading.
Concrete mismatch:

- In `/Users/abstractmapping/projects/landroid/testdata`, these files do exist:
  - `deskmap-stress-5x200.import.csv`
  - `deskmap-stress-5x200.README.md`
  - `deskmap-stress-5x200.summary.json`
  - `deskmap-stress-5x200.workspace.json`
  - `split-tract-imports-5x200/...`
- In that same folder, these files do **not** exist at all:
  - `upload-assorted-200.import.csv`
  - `upload-variant-512.import.csv`
  - `upload-variant-1024.import.csv`

So if Finder is showing those `upload-*` files, you are looking at a different copy of the project or Finder is showing a stale window.
Also, the files I changed in this phase have newer mtimes on disk:

- sw.js — Mar 12, 2026 10:37
- src — Mar 12, 2026 10:43
- index.html — Mar 12, 2026 10:44
- package.json — Mar 12, 2026 10:43
- dist/app.js — Mar 12, 2026 10:44

Why `scripts` and `testdata` still look old:

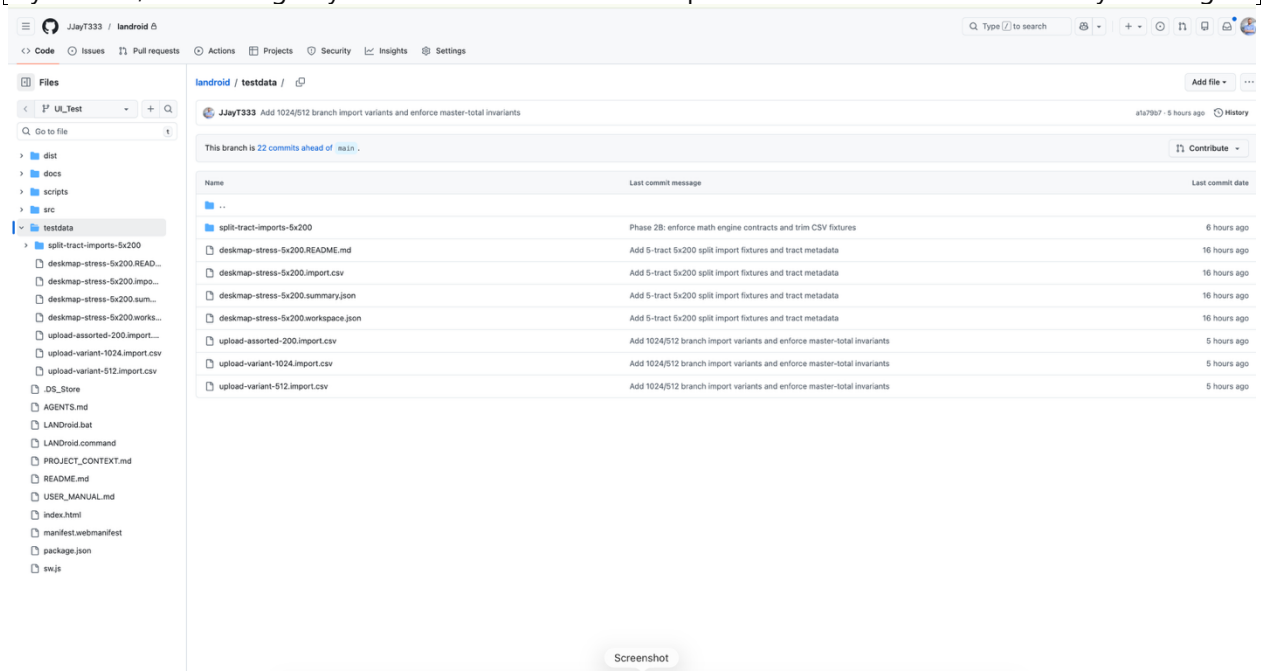
- Folder modified dates usually change when entries are added/removed/renamed.
- They do not reliably reflect file-content edits elsewhere in the repo.
- I did not touch `testdata` in this latest phase.

Fastest verification:

1. In Finder, press `Cmd+Shift+G`.
2. Paste `/Users/abstractmapping/projects/landroid`.
3. Open testdata.

4. If you still see the `upload-*` files, you are not in the same working copy I'm operating on.

If you want, I can next give you a one-command check to prove which folder Finder is actually showing.



Its in there

Worked for 44s

It is in `origin/UI_Test`. It is not in the branch that is currently checked out on disk.

Concrete state:

- Local working branch: `exp/ui-ideas-2026-03-12` at `ceecf81`
- GitHub screenshot branch: `origin/UI_Test` at `27b4b6d`
- `origin/UI_Test` contains these files in testdata:
  - `upload-assorted-200.import.csv`
  - `upload-variant-512.import.csv`
  - `upload-variant-1024.import.csv`
- Your current local branch does not contain those files; it contains the `split-tract-imports-5x200/*` fixtures instead.

So the mismatch is branch state, not Finder. The browser screenshot is showing one branch, and the files on your machine are from another branch.

One more constraint: the current local checkout has uncommitted blank-screen fixes in it. That means I should not blindly switch branches without handling those changes first.

I can take either next step:

- bring the `UI_Test` `testdata` files into the current branch
- create/switch to a local `UI_Test` checkout after preserving the current changes
- diff `UI_Test` vs the current branch and show you exactly what diverged

Model changed from GPT-5.4 to GPT-5.3-Codex.

bring those files into this branch. the screen works now. but there are a lot of issues with zooming. After bringing in those files. i will tell you what the next steps are

Worked for 49s

Done. I brought those files into your current branch from `origin/UI_Test`:

- upload-assorted-200.import.csv
- upload-variant-512.import.csv

- upload-variant-1024.import.csv

Verification:

- `testdata` now lists all three files.
- Git shows them as new in this branch (`??`) under `testdata/`.

Ready for your next zooming fixes.

Model changed from GPT-5.3-Codex to GPT-5.4.

Read the entire codebase first and do not make any changes until the full analysis is complete. This task has three goals, in this exact order of priority: 1. Audit and verify the correctness of the mineral ownership math engine. 2. Diagnose the zooming and scrolling performance issue when large datasets are present. 3. Perform a complete technical audit of the codebase including architecture, performance, tooling, and scalability. Think and review the project like: - a senior software architect - a performance engineer - a reliability engineer - a code reviewer - a product-minded technical lead Your job is not just to patch issues but to determine whether the current implementation is fundamentally correct and scalable. ---------- PART 1 — Math Engine Audit Find and examine the logic responsible for all ownership and interest calculations. Identify: - the files that contain the mineral interest calculation engine - functions that compute ownership percentages - how fractional interests are propagated through chains of conveyances - how totals are aggregated Verify mathematical correctness. Pay special attention to situations where cumulative interests exceed 100%. A correct ownership chain must always follow this multiplication pattern: I_{final} = I_{original} \times f_1 \times f_2 \times f_3 \times \dots Where: - \(I_{original}\) is the original grantor interest - each \(f\) represents the fraction conveyed in each transaction Example structure: Grantor → 1/2 Each of those → 1/4 Each of those → 1/8 Total owners increase while individual interest decreases. The engine must ensure that: • ownership propagation is mathematically correct • fractional multiplication is applied properly • cumulative totals never exceed the original ownership base Investigate: - floating point precision issues - rounding errors - incorrect aggregation logic - duplicated allocation - incorrect parent-child propagation - incorrect normalization - cases where ownership is added instead of multiplied Provide: 1. Exact files responsible for the math engine 2. Explanation of the current calculation approach 3. Whether the math is correct or flawed 4. Specific cases where totals could exceed 100% 5. The safest fix if problems exist 6. Whether the calculation engine architecture should be redesigned Do not modify files yet. ---------- PART 2 — Functional and Logic Review Identify what the application currently does and how major workflows operate. Find: - incomplete features - fragile assumptions - hidden bugs - logical inconsistencies - edge cases likely to break with larger datasets Explain which modules control core functionality. ---------- PART 3 — Architecture Review Explain the structure of the codebase and how responsibilities are divided across modules. Identify: - tight coupling - duplicated logic - poor separation of concerns - overly complex flows - brittle design patterns Determine whether the architecture is appropriate for future growth. ---------- PART 4 — Performance and Efficiency Review Identify performance bottlenecks such as: - repeated expensive calculations - unnecessary renders - inefficient loops - memory-heavy structures - blocking work on the main thread - layout thrashing - large DOM or SVG trees Determine whether the application will scale with significantly larger datasets. Prioritize fixes by impact. ---------- (HIGHEST PRIORITY!!!)PART 5 — Zoom and Scroll Performance Diagnosis Investigate the zoom and scroll behavior with large datasets. Identify: - which components handle zooming - which components handle scrolling - event listeners and handlers involved - rendering pipelines - state updates triggered during interactions Look for causes such as: - too many rendered elements - expensive redraws during zoom - repeated recalculation of layout - unthrottled scroll events - excessive state updates - layout reflows - inefficient data structures If the architecture fundamentally limits performance, determine whether it should be replaced. Possible improvements may include: - virtualization or windowing -

incremental rendering - memoization - debounced or throttled handlers - requestAnimationFrame rendering loops - separating visible data from full dataset - canvas-based rendering instead of heavy DOM/SVG - caching layout calculations Provide solutions in this order: 1. smallest safe fix 2. moderate refactor 3. architecture redesign if required ---------- PART 6 — Technology Stack Review Evaluate the current language, framework, and libraries. Determine: - whether the current stack is appropriate - whether any dependencies are unnecessary or outdated - whether alternative libraries would improve performance or maintainability Do not recommend changes unless there is a strong practical reason. ---------- PART 7 — Library and Tooling Review Evaluate tooling for: - testing - linting - formatting - type safety - profiling - build systems - state management - performance monitoring - CI/CD - logging and debugging Recommend best-in-class tools only where justified. ---------- PART 8 — Code Quality Review Identify: - code smells - dead code - duplication - unclear naming - poor abstractions - confusing folder structures Explain where refactoring would improve maintainability. ---------- PART 9 — Testing and Reliability Review Evaluate the current testing approach. Identify missing tests and recommend: - unit tests - integration tests - end-to-end tests - regression tests - performance tests Highlight the most critical areas needing coverage. ---------- PART 10 — Output Format Before Editing Provide results in this structure: A. Executive Summary B. Math Engine Assessment C. Functional Issues D. Architecture Assessment E. Performance Assessment F. Zoom and Scroll Root Cause Analysis G. Stack / Library / Tooling Assessment H. Code Quality Assessment I. Testing / Reliability Assessment J. Security Assessment K. Prioritized Action Plan Action plan should include: - immediate fixes - high-value refactors - optional improvements - long-term redesign ideas ---------- Rules - Do not change files yet - Tie every recommendation to specific files or modules - Avoid vague advice - Preserve user-facing behavior - Do not create a pull request - Wait for approval before implementing changes

Worked for 13m 19s

A. Executive Summary

- No files were changed.
- The mineral ownership engine is not currently safe enough to call mathematically correct in all user paths. The core logic in src/app.jsx (line 791) can produce totals above the ownership base, accepts non-finite/negative inputs in some paths, and lets users directly edit derived values.
- The zoom/scroll lag is mainly architectural. Large interactions in src/app.jsx (line 1530), src/app.jsx (line 1964), and src/app.jsx (line 2145) rerender the full DOM/SVG graph on every interaction frame.
- The codebase is serviceable as a prototype, not as a scalable ownership platform. Source of truth is split across UI state, desk-map mirrors, duplicated test logic, and a built runtime bundle.

B. Math Engine Assessment

- The production ownership math lives in src/app.jsx (line 791) through src/app.jsx (line 1186). Persistence/hydration that affects math inputs lives in src/workspaceDomain.js (line 63) and src/workspaceDomain.js (line 278). Test mirrors live in scripts/title-scenario-checks.js, scripts/math-regression-checks.js, and scripts/large-deskmap-stress.js.
- The current model stores two mutable values per conveyance node: `initialFraction` as the original granted interest and `fraction` as the remaining retained interest. New conveyances compute `calcShare` in src/app.jsx (line 862), descendants are rescaled in src/app.jsx (line 849), and totals are aggregated only as a UI health check in src/app.jsx (line 342).

- The math is only conditionally correct. It is correct when the conveyed share is bounded by the parent's remaining interest and when ancestor-relative multiplication is used. It is flawed overall because the runtime does not enforce those conditions.
- Totals can exceed 100% in at least five concrete ways:
  - `splitBasis === 'whole'` or `conveyanceMode === 'fixed'` can compute a share larger than `parent.fraction` in src/app.jsx (line 867), then `handleCommit` clamps the parent to zero but still grants the full child share in src/app.jsx (line 1094).
  - Attach-conveyance can oversubscribe the destination in src/app.jsx (line 813); destination balance is clamped down in src/app.jsx (line 825) while the attached branch still receives the full scaled amount.
  - Predecessor and rebalance paths do not enforce `newInitialFraction <= oldInitialFraction + parent.fraction` in production code. They scale descendants in src/app.jsx (line 1106) and src/app.jsx (line 1028), then clamp the parent afterward.
  - Direct edit and add-chain expose raw `fraction` and `initialFraction` inputs in src/app.jsx (line 3356), and the edit path can simply spread those values into state in src/app.jsx (line 1073).
  - `calcShare` is not validated for finiteness or sign. A zero denominator or negative fixed amount can yield `Infinity` or negative shares through src/app.jsx (line 867) and src/app.jsx (line 1096).
- Floating-point drift exists because all math uses `Number`. That is manageable only if invariants are strict. Current tests tolerate root sums up to `1.05` in scripts/math-regression-checks.js (line 94), which is too loose for ownership correctness.
- The safest fix is not a rewrite first. First, hard-block invalid commits in src/app.jsx (line 1062), make `fraction` derived rather than user-editable, and reject non-finite or negative math inputs. After that, extract the engine into a pure module and move to exact fraction or fixed-point math.
- The engine should be redesigned medium-term. Keeping core mineral-interest math embedded in a React component is not auditable or scalable.

C. Functional Issues

- The Research Hub is incomplete. Large sections are hard-disabled in src/app.jsx (line 2553) and src/app.jsx (line 2586), and the visible tab is explicitly a placeholder in src/app.jsx (line 2577).
- The manual interest matrix is disconnected from title math. `ownershipInterests` are entered by hand in src/app.jsx (line 1448) and never derived from the conveyance tree.
- Home-screen import can kick the user out of Home even if the import is canceled, because src/app.jsx (line 2270) flips `showHome` before src/app.jsx (line 1248) completes its prompt flow.
- Flow edge deletion is broken. The edge click path calls `deleteSelectedFlowElement(edge.id)` in src/app.jsx (line 2154), but the implementation in src/app.jsx (line 2120) ignores the argument and deletes the selected node instead.
- Offline and sync UX is misleading. `offlineModeActive` is set from browser capability in src/app.jsx (line 577), but sw.js (line 1) unregisters itself and USER_MANUAL.md (line 172) is now stale.

D. Architecture Assessment

- src/app.jsx is a single monolith with domain math, storage, import/export, audit logging, zoom handlers, and rendering all mixed together.

- Active desk-map state is duplicated between `nodes/pz` and `deskMaps[].nodes/pz`, with synchronization effects in src/app.jsx (line 421), src/app.jsx (line 445), and src/app.jsx (line 455). That is brittle and expensive.
- Test logic reimplements the engine instead of importing it from production. That weakens confidence because the runtime and the tests can drift independently.
- The browser actually runs dist/app.js from index.html (line 181), but the test suite in package.json (line 8) never proves that built artifact matches source.

E. Performance Assessment

- Desk Map panning writes React state on every pointer move in src/app.jsx (line 1536), so the full recursive tree renderer in src/app.jsx (line 1600) is revisited continuously.
- Flow Chart is worse. Pan, wheel, box drag, and tree drag all write state continuously in src/app.jsx (line 1988), src/app.jsx (line 2016), and src/app.jsx (line 2065).
- Edge rendering is O(E*N) per frame because each edge does two linear `find` calls in src/app.jsx (line 2145). At 1000 nodes and 990 edges, that is already expensive before painting.
- Tree/layout builders are also quadratic. `buildTree` in src/app.jsx (line 1508) and `buildFlowLayoutFromNodes` in src/app.jsx (line 1772) both repeatedly scan full arrays.
- None of the major views use windowing or viewport culling. The runsheet table in src/app.jsx (line 2470), desk map cards in src/app.jsx (line 1600), and flow nodes in src/app.jsx (line 2173) all render the full dataset.
- The current perf benchmark passed, but scripts/perf-benchmark-5x200.js measures data derivation only. It does not cover DOM size, paint cost, or interaction latency.

F. Zoom and Scroll Root Cause Analysis

- Desk Map zoom/pan is handled in src/app.jsx (line 1530) and src/app.jsx (line 1550). Flow Chart zoom/pan/drag is handled in src/app.jsx (line 1964), src/app.jsx (line 2016), and src/app.jsx (line 2042).
- The biggest zoom correctness bug is dual scaling. Flow Chart applies `flowPz.scale` in src/app.jsx (line 2805) and `treeScale` again in src/app.jsx (line 2835), but `fitFlowToView` sets both to the same value in src/app.jsx (line 1834). That squares the effective scale.
- Flow wheel zoom is unthrottled. Desk Map batches wheel deltas with `requestAnimationFrame` in src/app.jsx (line 1559); Flow Chart does not in src/app.jsx (line 2016).
- The performance root cause is full rerender, not raw math cost. Large pan/zoom interactions are paying for DOM diff, SVG edge lookup, and repaint on every frame.
- Smallest safe fix: add a memoized `flowNodeById`, RAF-batch Flow Chart pan/wheel, keep live transforms in refs during interaction and commit once on idle/up, and stop `fitFlowToView` from changing both scale layers.
- Moderate refactor: isolate Desk Map and Flow Chart into separate components, precompute adjacency maps, memoize node renderers, and window the runsheet.
- Architecture redesign: if the target is multi-thousand-node maps, move graph rendering to canvas or a hybrid DOM/canvas layer. The current rich DOM/SVG approach will cap out.

G. Stack / Library / Tooling Assessment

- React 19, esbuild, and PapaParse are reasonable choices for this product. I do not see a strong practical reason to replace the stack.
- The remaining reliability gap is delivery and guardrails, not framework selection. index.html (line 10) still depends on CDN Tailwind and Google Fonts, so the app is not fully self-contained.

- There is no repo-owned ESLint, Prettier, TypeScript, Playwright, Cypress, or CI configuration visible in the workspace.
- The build/test contract is weak. package.json (line 7) builds the runtime bundle, but package.json (line 8) does not verify that the shipped browser bundle is current.

H. Code Quality Assessment

- The core code smell is concentration: src/app.jsx is too large and too multi-purpose to change safely.
- There is dead or dormant code. `showCloudModal` exists but has no rendered UI path in src/app.jsx (line 271), and large research sections are hard-disabled.
- Some UX surface is effectively placeholder-only, such as "Move as tree group" in src/app.jsx (line 2775), which is always checked and read-only.
- Startup/storage failure messages at the top of src/app.jsx still reference old script-loading paths that no longer match the bundled boot path.

I. Testing / Reliability Assessment

- I ran and verified: `npm run test:smoke`, `npm run test:storage`, `npm run test:math-regression`, `npm run test:title-scenarios`, `npm run test:cross-surface`, and `npm run test:perf`. All passed.
- The missing tests are more important than the passing ones: there is no runtime test that invalid convey/rebalance/edit inputs are rejected, no browser test for pan/zoom smoothness, and no build-parity test for dist/app.js.
- The math tests bias inputs into safe ranges. For example, scripts/math-regression-checks.js (line 158) explicitly caps `newInitialFraction`, but the production UI does not.
- src/workspaceDomain.js (line 110) sanitizes self-parent and missing-parent cases, but there is still no full cycle validation on imported or loaded workspace graphs.

J. Security Assessment

- I did not find secrets, auth logic, or server-side trust boundaries in this repo. The main risks are import and browser-surface issues.
- CSV import trusts raw `INTERNAL_DOC` values in src/app.jsx (line 1317), and the viewer then loads them into `<object>` and outbound links in src/app.jsx (line 3448).
- `target="_blank"` is missing `rel="noopener noreferrer"` in src/app.jsx (line 3452).
- Audit and sync logs in src/auditLog.js (line 21) and src/syncEngine.js (line 21) are useful for UX only; they are not trustworthy audit evidence.
- `dangerouslySetInnerHTML` is still used in src/app.jsx (line 2259). It is currently constrained, but it remains a seam worth keeping narrow.

K. Prioritized Action Plan

- Immediate fix: enforce hard ownership invariants in src/app.jsx (line 1062) and remove manual editing of derived `fraction`.
- Immediate fix: correct Flow Chart zoom math by separating viewport zoom from print scale in src/app.jsx (line 1834) and src/app.jsx (line 2805).
- Immediate fix: replace `flowNodes.find` edge lookups with a memoized node map in src/app.jsx (line 2145), and RAF-batch Flow Chart wheel/pan handlers.
- High-value refactor: extract a pure ownership engine from src/app.jsx (line 791) and make scripts/title-scenario-checks.js and scripts/math-regression-checks.js import it instead of cloning logic.
- High-value refactor: remove mirrored active-map state in src/app.jsx (line 421) and src/app.jsx (line 455).

- Optional improvement: either finish or remove the dormant Research Hub surface in src/app.jsx (line 2528).
- Long-term redesign: if LANDroid is meant to become an auditable mineral-ownership platform, move to a pure transaction graph with exact fraction arithmetic and a renderer that can cull or paint large graphs without full React rerenders.

If you want the next step to be implementation, I would start with the immediate math guards and the Flow Chart scale/rerender fixes first.

Model changed from GPT-5.4 to GPT-5.3-Codex.

I am going to move this to a new window. Create a prompt that outlines everything we just went over along with priority, persona, explicit instructions, etc. Everything we need to start. Also answer this: Can you reference other threads when working on one?

Worked for 13s

No. I cannot automatically read or reference other threads while working in one thread.
If you want continuity, paste the needed context (or this prompt) into the new window.

```
You are my implementation partner for LANDroid.

Repository:
- /Users/abstractmapping/projects/landroid
- Current local branch: exp/ui-ideas-2026-03-12
- There are existing uncommitted changes in this working tree. Do not revert
unrelated changes.

Operating mode:
- Think and review like:
  - senior software architect
  - performance engineer
  - reliability engineer
  - code reviewer
  - product-minded technical lead
- Be methodical, phased, incremental.
- Prioritize correctness and maintainability over speed.
- Preserve user-facing behavior unless explicitly approved.

Mandatory instruction precedence:
1) System/developer/user chat instructions
2) AGENTS.md
3) PROJECT_CONTEXT.md

Before any architectural decisions:
- Read PROJECT_CONTEXT.md.

Hard constraints:
- Work only inside this repository.
- Do not modify files yet until full analysis is complete.
- No PR creation.
- Tie every recommendation to specific files/lines.
- Avoid vague advice.
- Preserve existing workflows unless change is required and called out.

Primary goals (in this order):
1. Audit/verify mineral ownership math correctness.
```

2. Diagnose zoom/scroll performance issues on large datasets.
3. Complete technical audit: architecture, performance, tooling, scalability, reliability, security.

Known current architecture facts:
- Main runtime is monolithic: src/app.jsx (~3.4k LOC).
- Runtime boot uses bundled dist/app.js from index.html.
- Math logic currently lives inside src/app.jsx, not a separate pure domain module.
- Persistence/hydration normalization in src/workspaceDomain.js.
- Validation scripts in scripts/*.js.
- Service worker currently self-unregisters (sw.js), while USER_MANUAL still claims app-shell caching.

Known high-priority findings to verify and refine:
- Math engine is not fully guarded against invalid states:
  - Share can exceed parent remaining interest in convey/attach/rebalance/precede flows.
  - Some paths clamp parent after the fact, allowing over-allocation.
  - Direct edits allow overriding derived fractions.
  - Non-finite/negative input handling is inconsistent.
- Zoom/scroll lag root causes:
  - Full React re-render on high-frequency pointer/wheel updates.
  - O(E*N) edge lookup pattern in flow rendering (`find` per edge endpoint).
  - No virtualization/windowing for large lists/graphs.
  - Flow view uses two scale layers and fit logic can compound scale.
- Functional issues:
  - Research Hub partially disabled with placeholder-only UI.
  - Flow edge delete path appears to call node-delete logic.
  - Some UX and docs are stale/misaligned with current runtime behavior.
- Tooling gaps:
  - No repo-level lint/typecheck/CI config visible.
  - Tests are script-based and mostly validate derivations, not interactive render performance.

Required deep-dive scope:

PART 1 — Math engine audit
- Locate every file/function involved in ownership math:
  - ownership percentage computation
  - fractional propagation through parent-child conveyance chains
  - branch correction paths (precede, attach, rebalance)
  - aggregate totals logic
- Verify multiplication semantics:
  I_final = I_original * f1 * f2 * f3 * ...
- Identify where totals can exceed 100%.
- Check precision/rounding/non-finite handling.
- Check cycle/parent integrity and normalization assumptions.
- Output:
  1) exact files
  2) current approach
  3) correctness verdict
  4) failure cases
  5) safest fix

```
  6) redesign recommendation or not

PART 2 — Functional/logic review
- Identify incomplete features, hidden bugs, fragile assumptions, edge cases.

PART 3 — Architecture review
- Assess module boundaries, coupling, duplication, separation of concerns,
scalability fitness.

PART 4 — Performance review
- Identify bottlenecks:
  - expensive recalcs
  - render churn
  - inefficient loops/data structures
  - blocking main-thread work
  - large DOM/SVG costs
- Prioritize by impact.

PART 5 — Zoom/scroll root-cause diagnosis (highest practical urgency after
math correctness)
- Map full interaction pipeline:
  - components, listeners, state transitions, rendering path
- Determine smallest safe fix, moderate refactor, and architecture redesign
option.

PART 6 — Stack review
- Evaluate framework/libs appropriateness and dependency necessity.

PART 7 — Tooling review
- Testing, linting, formatting, typing, profiling, build, state management,
monitoring, CI/CD, logging.

PART 8 — Code quality review
- Smells, dead code, duplication, naming, abstractions, folder clarity.

PART 9 — Testing/reliability review
- Identify critical missing unit/integration/e2e/regression/performance
coverage.

PART 10 — Security review
- Input handling, injection surfaces, data safety, unsafe browser patterns,
trust boundaries.

Validation requirements:
- Run relevant checks and report exact commands/results.
- At minimum:
  - npm run test:smoke
  - npm run test:storage
  - npm run test:math-regression
  - npm run test:title-scenarios
  - npm run test:cross-surface
  - npm run test:perf

Output format (before any edits):
```

```
A. Executive Summary
B. Math Engine Assessment
C. Functional Issues
D. Architecture Assessment
E. Performance Assessment
F. Zoom and Scroll Root Cause Analysis
G. Stack / Library / Tooling Assessment
H. Code Quality Assessment
I. Testing / Reliability Assessment
J. Security Assessment
K. Prioritized Action Plan

Action plan must include:
- immediate fixes
- high-value refactors
- optional improvements
- long-term redesign ideas

After delivering analysis:
- Wait for approval before implementing code changes.
```

GPT-5.3-Codex

Medium

LocalDefault permissions

UI_Test

Terminal

zsh